Тема 13: Обектно-ориентирано програмиране. Основни принципи. Класове и обекти. Наследяване и капсулация

Анотация:

- Класове и обекти.
    - Декларация на клас и декларация на обект.
    - Абстракция със структури от данни.
    - Капсулация и скриване на информацията.
    - Методи - декларация, предаване напараметри, връщане на резултат.
    - Основни видове конструктори.
    - Управление на динамичната памет и ресурсите (“RAII”).
- Наследяване.
    - Производни и вложени класове.
    - Достъп до наследените компоненти.
    - Статични полета и методи.

Концептуални основи на ООП

Обектно-ориентирано програмиране (Въведение)

Обектно-ориентираното програмиране (ООП) е програмна парадигма, при която една програмна система се моделира като набор от обекти, които взаимодействат помежду си. За разлика от процедурния подход, при който програмата представлява последователен списък от инструкции, тук обектите са основните носители на логика и данни. Всеки обект е способен да получава съобщения, да обработва информация и да комуникира с други обекти.

Обектно-ориентираният подход позволява обединяването на данни и свързани с тях операции в структури, наречени класове, от които се създават обекти. Това води до по-добра организация на кода, повторна употреба, модулност и разширяемост, като същевременно улеснява четимостта и поддръжката на програмата.

Основни принципи на обектно-ориентираното програмиране

Основните принципи на обектно-ориентираното програмиране - инкапсулация, абстракция, наследяване и полиморфизъм - представляват концептуалната рамка, чрез която се моделират сложни системи под формата на взаимодействащи си обекти с ясно дефинирано поведение и структура.

Капсулация

Капсулацията представлява скриване на вътрешното състояние на обекта от външния свят. Данните на обекта се предпазват от директна намеса, като достъпът до тях се осъществява

чрез публични методи (интерфейс). Това позволява контрол над начина, по който обектите се използват, и защитава тяхната цялост.

В C++ капсулацията се реализира чрез дефиниране на класове или структури, като се използват модификатори на достъп:

- private - членовете са достъпни само от методите на същия клас;
- protected - достъпни са от класа и неговите наследници;
- public - достъпни са от всяка точка на програмата. Важно е да се отбележи, че:

При class членовете по подразбиране са private; При struct членовете по подразбиране са public;

Абстракция

Абстракцията представлява концепция, при която на потребителя на даден обект се предоставя само необходимата за работа информация, докато вътрешната реализация остава скрита. По този начин се намалява сложността на системата, улеснява се използването ѝ и се подобрява поддръжката на кода. Абстракцията позволява програмистът да дефинира „какво прави“ даден обект, без да разкрива „как го прави“.

Наследяване

Наследяването е механизъм, чрез който се създават нови класове (наследници), базирани на вече съществуващи класове (базови). Базовият клас дефинира общи характеристики, които автоматично се наследяват от производния клас. Така се постига повторна употреба на код и организация на програмата в йерархия от класове.

Полиморфизъм

Полиморфизмът е принципът, който позволява дефиниране на унифициран интерфейс за различни типове обекти. С други думи, едни и същи действия се реализират по различен начин в зависимост от обектите, върху които се прилагат. (Това е кратко въведение, тъй като темата се разглежда в дълбочина по-късно).

Декларация на клас и декларация на обект

Класът в обектно-ориентираното програмиране представлява шаблон, по който се създават конкретни инстанции. Той дефинира структура (член-данни) и поведение (член-функции). Декларацията на клас сам по себе си не заделя памет.

```
// Декларация на клас
class Person {
private:
    std::string name;
    int age;

public:
    // ... методи
};
Обектът е конкретна инстанция на даден клас. При създаване на обект се заделя памет за неговите член-данни. Всеки обект има собствено състояние, но споделя дефиницията на поведението (методите) с другите обекти от същия клас.
```

```c++
// Декларация (и дефиниция) на обект
Person p1;
Person p2;
```

Обхват на видимост (Scope)

Класовете могат да бъдат дефинирани в различен обхват:

- Глобален обхват (на ниво namespace): Класът е достъпен в цялата програма след своята декларация.
- Локален обхват: Клас, дефиниран в тялото на функция, е достъпен само в нея (неговите методи трябва да бъдат дефинирани inline в тялото на класа).

Абстракция със структури от данни

Абстракцията е принцип, при който на потребителя на даден обект се предоставя само необходимата за работа информация (интерфейс), докато вътрешната реализация остава скрита. По този начин се намалява сложността и се улеснява използването. Класът е основният механизъм за абстракция в C++.

Абстракцията позволява програмистът да дефинира „какво прави“ даден обект, без да разкрива „как го прави“. Това се постига чрез:

- Интерфейс: Набор от public методи, чрез които се осъществява взаимодействие с обекта.
- Скрита реализация: Вътрешната логика и данните (private членове) не са достъпни директно.

Абстракция чрез капсулация (в рамките на един клас)

Това е най-фундаменталната форма на абстракция. Всеки добре написан клас, който скрива данните си (private) и предоставя публични методи за работа с тях, използва този принцип. Потребителят на класа не знае (и не трябва да знае) как точно се съхраняват данните или каква е вътрешната логика на методите.

```
class BankAccount {
public:
    // Конструктор, който задава начално състояние
    BankAccount(double initial_balance) {
        if (initial_balance >= 0) {
            balance = initial_balance;
        } else {
            balance = 0;
        }
    }

    // Публичен интерфейс - "какво прави" обектът
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    bool withdraw(double amount) {
```

```
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }

    double getBalance() const {
        return balance;
    }

private:
    // Скрита реализация - "как го прави"
    double balance;
    // Тук може да има и други скрити данни:
    // std::vector<Transaction> history;
    // double interest_rate;
};
```

Потребителят на този клас може да депозира, тегли и проверява наличност, без да има представа как се съхранява балансът, дали има допълнителни полета за лихви, такси или история на транзакциите. Сложността е скрита.

В този пример, BankAccount абстрахира идеята за банкова сметка.

Абстракция чрез полиморфизъм (за йерархия от класове)

Това е по-специализирана форма на абстракция, която се използва, когато искаме да дефинираме общ интерфейс (договор) за цяло семейство от класове. Тук се използват абстрактни класове с чисти виртуални функции.

Целта е да се гарантира, че всички наследници ще предоставят своя реализация на определено поведение, което позволява да се работи с тях по унифициран начин (полиморфизъм).

Капсулация и скриване на информацията

Капсулацията е механизъм за обединяване на данни и методи, които работят с тях, в единна структура (клас), като същевременно се скрива вътрешното състояние на обекта от външния свят.

В C++ капсулацията се реализира чрез модификатори на достъп:

- private: Членовете са достъпни само от методите на същия клас. По подразбиране в class.
- protected: Достъпни са от класа и неговите наследници.
- public: Достъпни са от всяка точка на програмата. По подразбиране в struct. Практическо приложение на капсулацията е използването на аксесори (getters) и мутатори (setters) – публични методи за контролиран достъп до private данни.

```
class Person {
private:
    int age;
public:
    int getAge() const { return age; } // Аксесор (Getter)
```

```
    void setAge(int a) { if (a > 0) age = a; } // Мутатор (Setter) с валидация
};
```

Методи - декларация, предаване на параметри, връщане на резултат

Методите (член-функциите) дефинират поведението на обекта.

- Декларация: Декларират се в тялото на класа.
- Указателят this: Всеки нестатичен метод получава скрит параметър this – указател към обекта, за който е извикан. Чрез него методът достъпва член-данните на конкретната инстанция.
- Константни методи: Метод, деклариран с const накрая, обещава да не променя състоянието на обекта. В такъв метод this е указател към константен обект и могат да се променят само mutable член-данни.

```
// Този код:
bool Point::isInFirstQuadrant() {
    return x >= 0 && y >= 0;
}
pt.isInFirstQuadrant();

// Вътрешно се превежда до нещо подобно на:
bool Point_isInFirstQuadrant(Point* const this) {
    return this->x >= 0 && this->y >= 0;
}
Point::isInFirstQuadrant(&pt);
```

Константни член-функции:

- Не променят член-данните на структурата.
- Оказва се чрез записването на ключовата дума const в декларацията и в края на заглавието(суфикс) в дефиницията им
- Могат да се извикват от константни обекти.

```
struct obj {
    void inspect() const;   //This member-function promises not to change *this
    void mutate();          //This member-function might change *this
};

void Test(obj& changeable, const obj& unchangeable) {
    changeable.inspect();   // Okay: doesn't change a changeable object
    changeable.mutate();    // Okay: changes a changeable object

    unchangeable.inspect(); // Okay: doesn't change an unchangeable object
    unchangeable.mutate();  // ERROR: attempt to change unchangeable object
}
```

- const индикира, че this е указател към const.
- Член-функции, които използват const по този начин не могат да променят обекта, върху който са извикани!

Основни видове конструктори

Конструктор

Конструкторът е специална член-функция на класа, която се извиква автоматично при създаването на обект от този клас. Основната му задача е да инициализира член-данните на обекта и да установи валидно начално състояние (класов инвариант), като гарантира, че обектът е напълно готов за употреба веднага след създаването си. Той носи същото име като класа и няма тип на връщана стойност.

Конструктор по подразбиране (Default Constructor)

Конструктор, който може да бъде извикан без никакви аргументи. Той се използва, когато обект се създава без изрично подадени начални стойности и е задължителен за някои операции, като например създаването на масиви от обекти. Компилаторът може да го генерира автоматично (като implicit default constructor), ако в класа не са дефинирани никакви други конструктори от потребителя.

- Сигнатура: `Widget();`

Параметризиран конструктор (Parameterized Constructor)

Формално обяснение: Всеки конструктор, който приема един или повече аргументи, за да инициализира член-данните на обекта с конкретни, подадени от потребителя стойности. Това е най-често срещаният тип конструктор, тъй като позволява гъвкаво създаване на обекти в различно начално състояние.

- Сигнатура: `Widget(int size, const std::string& name);`

Забележка: Съвпадащи имена на параметри и членове (Shadowing)

В случай че име на параметър съвпада с име на член-данна, той го "засенчва" (shadowing). За да се разреши тази двусмислица, може да се използва `this` указателят в тялото на конструктора. По-идиоматичният и ефективен (C++) подход обаче е чрез инициализиращия списък, който извършва директна инициализация.

```
class A {
private:
    int size;
public:
    // Предпочитан начин: Инициализиращ списък.
    // Еднозначен, ефективен и извършва истинска инициализация.
    A(int size) : size(size) {}
}

class B {
private:
    int size;
public:
       // Алтернатива: 'this' указател. Работи, но е по-малко ефективен, тъй
като извършва присвояване, а не инициализация.
       B(int size) {
           this->size = size; // 'this->size' е членът, 'size' е параметърът
       }
}
```

Копиращ конструктор (Copy Constructor)

Конструктор, който създава нов обект като копие на друг, вече съществуващ обект от същия клас. Той е от ключово значение за правилното управление на ресурси (например при динамична памет), за да се извърши "дълбоко копиране" (deep copy) и да се избегнат проблеми като двойно освобождаване на памет. Извиква се автоматично при инициализация на обект от друг, както и при подаване или връщане на обект по стойност от функция.

- Сигнатура: `Widget(const Widget& other);`

Преместващ конструктор (Move Constructor)

Конструктор, който ефективно "откраднва" ресурсите (като указатели към динамична памет или файлови манипулатори) от временен, изтичащ обект (rvalue) и ги прехвърля на новосъздадения обект. Вместо да извършва скъпо копиране на ресурси, той просто премества собствеността върху тях, което е значителна оптимизация на производителността, особено при връщане на обекти от функции. След преместването, оригиналният временен обект остава във валидно, но неопределено (обикновено "празно") състояние.

- Сигнатура: `Widget(Widget&& other) noexcept;`
    - Пояснение: Спецификаторът noexcept е критично важен, защото уверява стандартната библиотека, че преместването е безопасна операция и гарантирано няма да хвърли изключение

Допълнително: Конвертиращ конструктор (Converting Constructor)

Конвертиращ конструктор (Converting Constructor) е всеки конструктор с един аргумент, който дефинира правило за автоматично (имплицитно) преобразуване от друг тип към типа на класа. Нека разгледаме следния пример:

```
class foo {
public:
    // Това е конвертиращ конструктор от int към foo
    foo(int) {}
};

// Тази функция очаква обект от тип foo, подаден по стойност
void g(foo obj) {
    // ...
}
```

В него, при g(5) компилаторът сам създава обект foo от цялото число 5. За да се предотвратят нежелани конверсии и потенциални грешки, този конструктор може да се маркира с ключовата дума explicit, което забранява автоматичното преобразуване и изисква изрично извикване като g(foo(5))

Деструктор

Деструкторът е специална член-функция на класа, която се извиква автоматично, непосредствено преди обектът да бъде унищожен и паметта, която заема, да бъде освободена. Неговото основно предназначение е да освободи всички външни ресурси (като динамично заделена памет, отворени файлове или мрежови връзки), които обектът е придобил през живота си, за да се предотвратят течове на ресурси.

- Деструкторът (~Widget();) се извиква автоматично при унищожаване на обект и се използва за освобождаване на заети ресурси.

Инициализиращ списък (Member Initializer List)

Инициализиращият списък е специална синтактична конструкция в конструктора на класа, която позволява директна инициализация на член-данните на обекта, както и извикване на конструктори на базови класове. Този списък се изпълнява преди тялото на конструктора и е предпочитаният, а в някои случаи и единствено възможен начин за конструиране на обекта.

- Синтаксис и примери
    - Списъкът започва с двоеточие (:) след сигнатурата на конструктора и преди отварящата скоба { на тялото му
    - Инициализация на член-данни

```
class MyClass {
int x;
const int y;

public:
    MyClass(int val_x, int val_y)
        : x(val_x),   // Инициализира x със стойността на val_x
        y(val_y)    // Инициализира константата y със стойността на val_y
    {
        // Тялото на конструктора може да е празно.
    }
};
```

    - Извикване на конструктор на базов клас (при наследяване)
        - Това е основният механизъм, по който конструкторът на класа-наследник може да подаде аргументи към конструктора на своя родител.

```
#include <iostream>

class Base {
    int base_value;
public:
    // Базовият клас има само параметризиран конструктор
    Base(int val) : base_value(val) {
        std::cout << "Извикан е конструкторът на Base с_тойност: " <<
base_value << std::endl;
    }
};

class Derived : public Base {
    int derived_value;
public:
    // Конструкторът на Derived трябва да извика конструктора на Base
    Derived(int d_val, int b_val)
        // Синтаксис за извикване на базов конструктор:
        : Base(b_val),
        derived_value(d_val)
    {
        std::cout << "Извикан е конструкторът на Derived с_тойност: " <<
derived_value << std::endl;
    }
};

int main() {
```

```
    // При създаването на 'd' първо ще се извика конструкторът на Base,
    // а след това този на Derived.
    Derived d(100, 50);
    return 0;
}
```

Използването на инициализиращ списък е не просто добра практика, а е задължително в следните случаи:

- За извикване на конструктор на базов клас
    - Това е единственият начин да се подадат аргументи към конструктор на родителски клас.
    - Ако не го направите, компилаторът ще се опита да извика конструктора по подразбиране на базовия клас, което ще доведе до грешка, ако такъв не съществува.
- За const член-данни
    - Константите трябва да бъдат инициализирани при тяхното създаване.
- За членове, които са референции (&)
    - Референциите също трябва да бъдат свързани с обект в момента на тяхното деклариране.

За член-данни, които са обекти от класове без конструктор по подразбиране.

Освен това, този подход е по-ефективен, тъй като извършва директна инициализация, а не по-бавната комбинация от "създаване по подразбиране + последващо присвояване".

Управление на динамичната памет и ресурсите (“RAII”)

RAII (Resource Acquisition Is Initialization) е централен идиом в C++. Идеята е, че придобиването на ресурс (памет, файл, мрежова връзка) се обвързва с инициализацията на обект, а освобождаването на ресурса – с неговото унищожаване (в деструктора). Това гарантира автоматично и надеждно управление на ресурсите.

Разширените механизми за управление на ресурси при копиране и преместване са:

- Оператор за присвояване чрез копиране (Copy Assignment Operator)
    - Модифицира съществуващ обект, като копира ресурсите от друг.
    - Трябва да се пази от самоприсвояване и да управлява правилно паметта(`Widget& operator=(const Widget& other);`)
- Семантика на преместване (Move Semantics)
    - Осигурява ефективно прехвърляне на собствеността над ресурси от временни обекти, избягвайки скъпо копиране.
    - Реализира се чрез преместващ конструктор и оператор за присвояване чрез преместване (`Widget& operator=(Widget&& other) noexcept;`)

Дълбоко копиране: Копиращ конструктор срещу Оператор за присвояване

Когато един клас управлява ресурси (например динамична памет), генерираните от компилатора по подразбиране копиращи операции извършват т.нар. плитко копиране (shallow copy). Те просто копират стойностите на член-данните бит по бит. Ако член-данната е указател, ще се копира само адресът, а не паметта, към която той сочи. Това води до два сериозни проблема:

- Два обекта ще сочат към един и същ ресурс, което може да доведе до неочаквани промени.
- При унищожаване на обектите, деструкторът ще се опита да освободи един и същ ресурс два пъти, което води до неопределено поведение и най-често до срив на програмата (double-free error).

За да се избегне това, трябва да реализираме дълбоко копиране (deep copy), като дефинираме изрично копиращ конструктор и оператор за присвояване.

Основното правило е:

- Конструкторът се извиква, когато се създава нов обект.
- Операторът за присвояване се извиква, когато се модифицира вече съществуващ обект.

Нека разгледаме синтактичните форми:

```
Test t1; // Извиква се конструктор по подразбиране

// Форми, които извикват КОПИРАЩ КОНСТРУКТОР:
Test t2(t1);      // 1. Директна инициализация. Най-явната форма.
Test t3 = t1;     // 2. Копи-инициализация. Въпреки символа '=', тук се създава
                  //    нов обект t3, затова се вика конструктор.

// Форма, която извиква ОПЕРАТОР ЗА ПРИСВОЯВАНЕ:
t2 = t1;          // 3. Присвояване. Обектите t1 и t2 вече съществуват.
                  //    Състоянието на t1 се присвоява на t2.
```

Разбирането на разликата между инициализация (случаи 1 и 2) и присвояване (случай 3) е критично.

Правила за автоматично генериране (Rule of Zero/Three/Five)

Компилаторът може автоматично да генерира специалните методи (деструктор, копиращи и преместващи операции). Правилата гласят:

- Правило на нулата (Rule of Zero)
    - Ако класът не управлява ресурси директно, не дефинирайте нито един от специалните методи.
    - Компилаторът ще се справи отлично.
- Правило на тройката (стандарт до C++11)
    - Ако дефинирате един от {деструктор, копиращ конструктор, копиращ оператор=}, вероятно трябва да дефинирате и трите.

- Правило на петицата (C++11 и нагоре)
    - Ако дефинирате един от петте {деструктор, копиращи операции, преместващи операции}, трябва да се съобразите с всички пет.
    - Например, дефинирането на деструктор спира автоматичното генериране на преместващи операции.

Наследяване

Наследяването е механизъм в обектно-ориентираното програмиране, чрез който се създават нови класове (наследници), базирани на вече съществуващи класове (базови). Базовият (родителски) клас дефинира общи характеристики – член-данни и методи, които автоматично се наследяват от производния (наследник) клас. Така се постига повторна употреба на код и организация на програмата в йерархия от класове.

Производни класове

Производен клас (или наследник) е клас, който е създаден на базата на вече съществуващ клас - базов клас. Той наследява всички публични и защитени членове (променливи и методи) на базовия клас и може да добавя нови членове или да модифицира (презаписва) наследени методи.

```
class Base {
public:
    void foo() { std::cout << "Base foo\n"; }
};

class Derived : public Base {  // Derived е производен клас на Base
public:
    void bar() { std::cout << "Derived bar\n"; }
};
```

Вложени класове

Вложени класове са класове, дефинирани вътре в друг клас (външен клас). Те служат за логическа групировка и капсулиране на свързана функционалност. Вложеният клас има достъп до публичните и защитените статични членове на външния клас, но за достъп до нестатичните членове се нуждае от обект на външния клас.

Пример:

```
class Outer {
public:
    class Inner {
    public:
        void display() { std::cout << "Inner class\n"; }
    };
};

// Достъп до вложения клас
Outer::Inner obj;
obj.display();
```

Видове наследяване според достъпа:

В CPP се позволява три вида наследяване, които се определят от спецификатора за достъп:

```
class Derived : public Base { ... };     // публично наследяване
class Derived : protected Base { ... };  // защитено наследяване
class Derived : private Base { ... };    // частно наследяване
```

- При `public` наследяване
  - `public`и `protected` членовете на базовия клас запазват достъпността си.
- При `protected`
  - `public` и protected`членове стават`protected`.
- При `private`
  - всичко става `private`.

Пояснение: Типът наследяване по подразбиране зависи от ключовата дума, използвана за дефиниране на производния клас:

- При `class Derived : Base { ... }`; наследяването по подразбиране е `private`.
- При `struct Derived : Base { ... };` наследяването по подразбиране е `public`.

Важно е също така, че:

- Производният клас не може да достъпва директно `private` членове на базовия клас, но може да използва публични или защитени методи за достъп.
- Наследствените методи могат да бъдат презаписвани (override) ако са виртуални.

Пример за наследяване и разширяване на поведение

```
class Animal {
public:
    void eat() { std::cout << "Eating...\n"; }
};

class Dog : public Animal {
public:
    void bark() { std::cout << "Barking...\n"; }
};
// Класът Dog наследява метода eat() от Animal и добавя собствен метод bark().
```

(Допълнително*)Наследяване ("is-a") срещу композиция и агрегацията("has-a")

Композицията представлява „силна“ връзка между обектите: ако обект A съдържа обект B чрез композиция, то B не може да съществува независимо от A.

```
class Engine {
    // ...
};

class Car {
private:
```

```
    Engine engine; // композиция
};
// Тук engine е част от Car, създава се и се унищожава заедно с него.
```

Агрегацията е „по-слаба“ връзка, при която обектът B може да съществува независимо от обекта A. Обикновено се реализира чрез указатели или референции.

```
class Engine {
    // ...
};

class Car {
private:
    Engine* engine; // агрегация
public:
    Car(Engine* e) : engine(e) {}
};
// Тук Car използва Engine, но не го притежава – двигателят може да се използва
и от други обекти.
```

В обобщение, макар и наследяването, композицията и агрегацията да служат за преизползване на код, изборът между тях зависи от логическата връзка между обектите. Наследяването (връзка "is-a") е подходящо, когато даден клас е специализация на друг и се цели използване на полиморфизъм. От друга страна, композицията и агрегацията (връзка "has-a") се използват за изграждане на сложни обекти чрез включване на други като компоненти, като това често води до по-гъвкав дизайн и по-слаба степен на свързаност между класовете.

Статични полета и методи

Статичните членове (данни и методи) са асоциирани със самия клас, а не с конкретна инстанция (обект) на класа. Това означава, че съществува само едно-единствено копие на статичните данни, което се споделя между всички обекти от този клас.

- Статични данни (Static Data Members):
    - Декларират се с ключовата дума `static` вътре в класа, но задължително се дефинират и инициализират извън него, в глобалното пространство.
    - Пазят се в статичната/глобалната памет
        - съществуват през цялото време на изпълнение на програмата.
- Статични методи (Static Member Functions):
    - Не получават скрит this указател, тъй като не са свързани с конкретен обект.
    - Поради това могат да достъпват директно само други статични членове (данни или методи) на класа.
    - Извикват се чрез името на класа и оператора за обхват (`::`), например `ClassName::staticMethod()`.

```
#include <iostream>

class Widget {
public:
    Widget() {
        count++; // Увеличаваме общия брояч при създаване на всеки обект
```

```
    }
    ~Widget() {
        count--; // Намаляваме общия брояч при унищожаване на всеки обект
    }

    // Статичен метод за достъп до статичния брояч
    static int getCount() {
        return count;
    }

private:
    // Декларация на статичната член-данна
    static int count;
};

// Дефиниция и инициализация на статичната член-данна
int Widget::count = 0;

int main() {
    std::cout << "Начален брой: " << Widget::getCount() << std::endl; // 0

    Widget w1;
    Widget w2;
    std::cout << "Брой след създаване на w1 и w2: " << Widget::getCount() <<
std::endl; // 2

    {
        Widget w3;
        std::cout << "Брой след създаване на w3: " << w1.getCount() <<
std::endl; // 3 (достъп и през обект)
    } // w3 се унищожава тук

    std::cout << "Брой след унищожаване на w3: " << Widget::getCount() <<
std::endl; // 2
    return 0;
}
```

Статични членове и наследяване

Поведението на статичните членове при наследяване е ключово за разбиране: те не се дублират в класа-наследник.

- Статични данни
    - Съществува само едно копие на статичната променлива, дефинирана в базовия клас.
    - Всички инстанции на базовия клас и всички инстанции на неговите наследници споделят и променят същата тази променлива.
- Статични методи
    - Статичните методи на базовия клас могат да бъдат извикани и през името на класа-наследник.
    - Важно е да се отбележи, че те не могат да бъдат виртуални (не участват в полиморфизма).
    - Ако наследник дефинира статичен метод със същото име, той скрива (hides) метода на базовия клас, а не го предефинира (overrides).

```
#include <iostream>
```

```
class Entity {
public:
    // Брояч, общ за всички същества в играта
    static int entity_count;

    Entity() {
        entity_count++;
    }

    static int getEntityCount() {
        return entity_count;
    }
};

// Дефиниция на статичния член от базовия клас
int Entity::entity_count = 0;

// Player наследява Entity
class Player : public Entity {
public:
    std::string name;

    Player(std::string name) : name(name) {
        // Конструкторът на базовия клас (Entity) се извиква автоматично,
        // увеличавайки същия брояч entity_count.
        std::cout << "Създаден е играч: " << name << std::endl;
    }
};

int main() {
    std::cout << "Общо същества в началото: " << Entity::getEntityCount() <<
std::endl; // 0

    Player p1("Gandalf");
    Player p2("Aragorn");
    Entity monster;

    // Достъпваме ЕДНА И СЪЩА променлива през различните класове
    std::cout << "Общо същества (през Entity): " << Entity::getEntityCount() <<
std::endl; // 3
    std::cout << "Общо същества (през Player): " << Player::getEntityCount() <<
std::endl; // 3
    std::cout << "Общо същества (директен достъп през Player): " <<
Player::entity_count << std::endl; // 3
}
```